

System Detective

Linux System Inspection and Memory Locality Analysis

Jayachandra Vandanapu
TY-10 33284

1. Objective

The aim of this activity was to inspect a Linux system, perform a memory locality experiment, and display the results using a dashboard.

2. Experimental Environment

The experiments were performed inside WSL2 using Ubuntu 24.04.2 LTS.

Property	Value
Environment	WSL2
Operating System	Ubuntu 24.04.2 LTS
Architecture	x86_64
Kernel	5.15.167.4-microsoft-standard-WSL2
Compiler	GCC 13.3.0
Node.js	18.19.1

Table 1: Experimental environment

3. System Inspection

The following Linux commands were used to inspect the system:

- `lscpu` – CPU architecture, cores, threads and cache information.
- `free -h` – memory and swap information.
- `top` – current process and CPU activity.
- `ps -eLf` – processes and lightweight process information.
- `strace -c` – summary of system calls made by a command.

The `lscpu` output showed an x86_64 environment with 8 logical CPUs, 4 cores and 2 threads per core.

```

vanda@JAY-16CHANDRA:~/system-detective$ lscpu
Architecture:          x86_64
CPU op-mode(s):        32-bit, 64-bit
Address sizes:          39 bits physical, 48 bits virtual
Byte Order:             Little Endian
CPU(s):                 8
On-line CPU(s) list:    0-7
Vendor ID:              GenuineIntel
Model name:             11th Gen Intel(R) Core(TM) i5-1135G7 @ 2.40GHz
CPU family:             6
Model:                 140
Thread(s) per core:     2
Core(s) per socket:     4
Socket(s):              1
Stepping:               1
BogoMIPS:               4838.39
Flags:                  fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat pse36 clflush mmx fxsr sse s
se2 ss ht syscall nx pdpe1gb rdtscp lm constant_tsc arch_perfmon rep_good nopl xtopology tsc_r
eliable nonstop_tsc cpuid pni pclmulqdq vmx ssse3 fma cx16 pdcm pcid sse4.1 sse4.2 x2apic movb
e popcnt tsc_deadline_timer aes xsave avx f16c rdrand hypervisor lahf_lm abm 3dnowprefetch inv
pcid_single ssbd ibrs ibpb stibp ibrs_enhanced tpr_shadow vnmi ept vpid ept_ad fsgsbase tsc_ad
just bml avx2 smep bmi2 erms invpcid avx512f avx512dq rdseed adx smap avx512ifma clflushopt c
lwb avx512cd sha_ni avx512bw avx512vl xsaveopt xsavec xgetbv1 xsaves avx512vbmi umip avx512_vb
mi2 gfni vaes vpclmulqdq avx512_vnni avx512_bitalg avx512_vpopcntdq rdpid movdiri movdir64b fs
rm avx512_vp2intersect md_clear flush_l1d arch_capabilities

Virtualization features:
Virtualization:         VT-x
Hypervisor vendor:      Microsoft
Virtualization type:    full
Caches (sum of all):
L1d:                    192 KiB (4 instances)
L1i:                    128 KiB (4 instances)
L2:                     5 MiB (4 instances)
L3:                     8 MiB (1 instance)
Vulnerabilities:
Gather data sampling:    Unknown: Dependent on hypervisor status
Itlb multihit:          Not affected
L1tf:                   Not affected
Mds:                    Not affected
Meltdown:               Not affected
Mmio stale data:        Not affected
Reg file data sampling: Not affected
Retbleed:               Mitigation; Enhanced IBRS
Spec rstack overflow:    Not affected
Spec store bypass:      Mitigation; Speculative Store Bypass disabled via prctl and seccomp
Spectre v1:             Mitigation; usercopy/swapgs barriers and __user pointer sanitization
Spectre v2:             Mitigation; Enhanced / Automatic IBRS; IBPB conditional; RSB filling; PBRSB-eIBRS SW sequence;
                        BHI SW loop, KVM SW loop
Srbds:                  Not affected
Tsx async abort:        Not affected

```

Figure 1: CPU information obtained using `lscpu`.

The memory information was collected using `free -h`.

```

vanda@JAY-16CHANDRA:~/system-detective$ free -h
              total        used        free      shared  buff/cache   available
Mem:          3.7Gi       759Mi       1.2Gi       3.2Mi       2.0Gi       3.0Gi
Swap:         1.0Gi           0B       1.0Gi

```

Figure 2: Memory information obtained using `free -h`.

4. Memory Locality Experiment

A 4096×4096 two-dimensional array of `double` values was used for the experiment. Each element occupies 8 bytes, giving an array size of approximately 128 MiB.

Two traversal methods were compared:

- **Row-major traversal:** rows are traversed from left to right before moving to the next row.
- **Column-major traversal:** columns are traversed from top to bottom before moving to the next column.

The program was written in C and compiled using GCC with the `-O2` optimization level.

C stores two-dimensional arrays in row-major order. Therefore, row-major traversal accesses consecutive memory locations more frequently. Column-major traversal accesses elements that are farther apart in memory, resulting in poorer spatial locality.

5. Experimental Results

The locality experiment was executed three times. The following values are from the latest recorded benchmark run.

Run	Row-major (s)	Column-major (s)	Ratio
1	0.022200	0.252546	11.38×
2	0.020004	0.251715	12.58×
3	0.019275	0.254740	13.22×
Mean	0.020493	0.253000	12.35×

Table 2: Memory locality benchmark results

In all three runs, row-major traversal was faster than column-major traversal. The mean column-major execution time was approximately 12.35 times the mean row-major execution time.

The exact times changed slightly between runs, but the same overall pattern was observed.

```
vanda@JAY-16CHANDRA:~/system-detective/locality$ ./locality_c
Array size: 4096 x 4096
Total elements: 16777216
Element size: 8 bytes
Array size: 128.00 MiB

run,row_major_s,column_major_s,ratio
1,0.022200,0.252546,11.38
2,0.020004,0.251715,12.58
3,0.019275,0.254740,13.22

Mean row-major time: 0.020493 seconds
Mean column-major time: 0.253000 seconds
Mean ratio: 12.35x

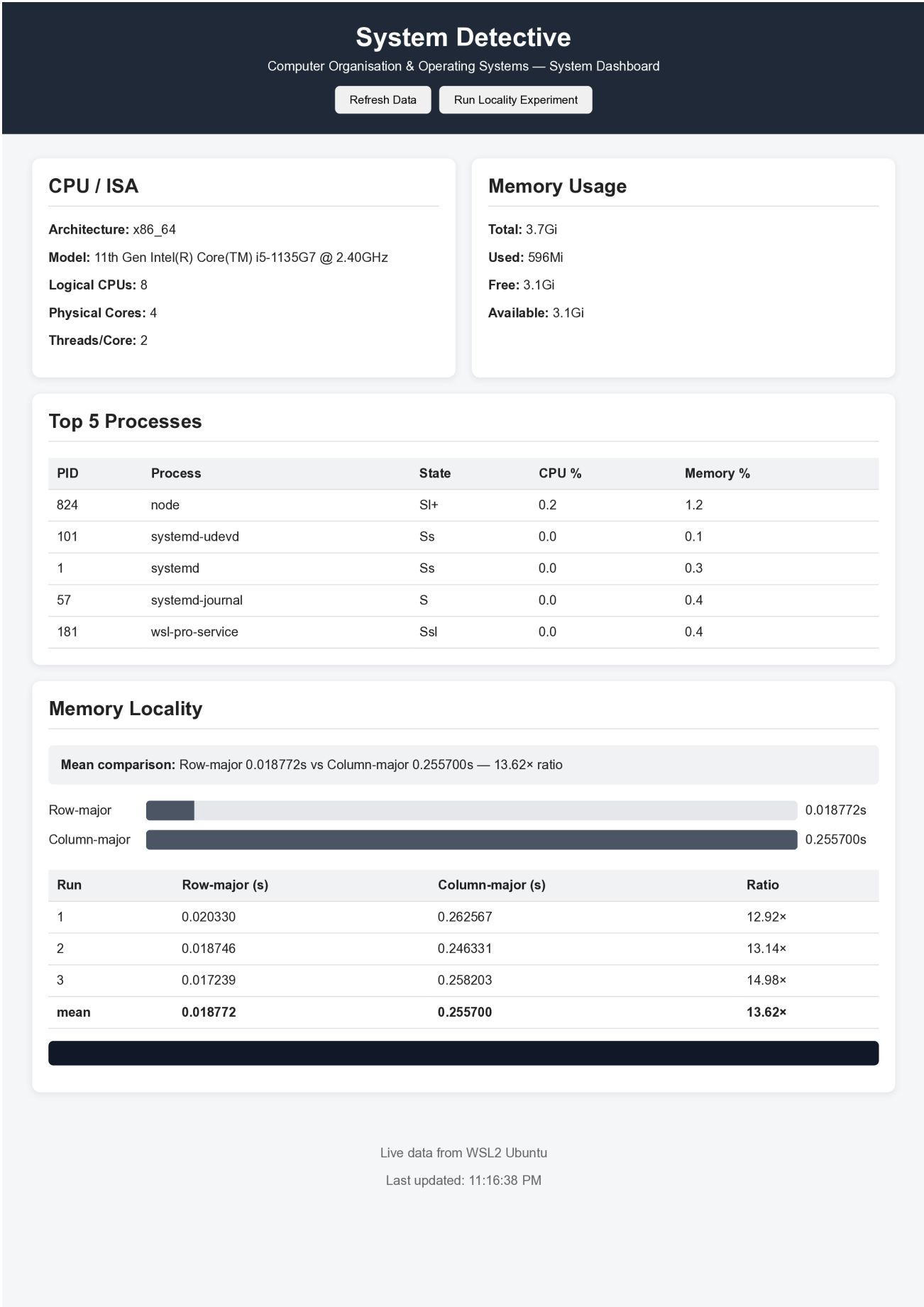
Results saved to results.csv
```

Figure 3: Locality experiment results.

6. System Dashboard

A simple dashboard was developed using HTML, CSS, JavaScript and Node.js. The dashboard displays CPU information, memory usage, the top five processes and locality results.

The backend obtains system information from Linux commands and reads the locality benchmark results. It can also run the locality experiment again.



7. AI Verification and Reproducibility

I used AI during the project for debugging, explanations and help with the dashboard. System-specific results were checked by running the commands on my own WSL2 system.

One AI-generated prediction was also tested experimentally. The prediction stated that the process with the highest CPU usage would have a CPU value greater than 0.0%. The following command was executed:

```
ps -eo pid,comm,stat,%cpu,%mem --sort=-%cpu | head -n 6
```

The recorded output showed 0.0% CPU usage for the listed processes, disproving the prediction. This showed that the AI prediction did not match the actual system output.

8. Conclusion

This activity helped me understand Linux system information, processes, system calls and memory locality. The locality experiment showed a clear difference between row-major and column-major access. I also used the dashboard to bring the collected system and benchmark information together in one place.